🔷 1️⃣ Derivation of Index Formula for 1D Array in C

✅ Basic Idea

In C, arrays are stored in **contiguous memory locations**.
Let:

- Base address of array = **B**
- Size of each element = **w** bytes
- Index of element = **i**
- Lower bound index = **LB** (in C, usually 0)

🔷 Memory Representation

If we have:
int A[5];
Assume:

- Base address (B) = 1000
- Size of int (w) = 4 bytes

Memory layout:

Index	Address
A[0]	1000
A[1]	1004
A[2]	1008
A[3]	1012
A[4]	1016

Each next element increases by **w bytes**.

🔷 Derivation

To reach element A[i]:

1. Number of elements before it = (i - LB)
2. Each element takes w bytes
3. Total offset = (i - LB) × w
4. Add offset to base address

✅ Final Formula:

$$\text{Address}(A[i]) = B + (i - LB) \times w$$

🔷 In C (LB = 0)

$$\text{Address}(A[i]) = B + i \times w$$

🔷 Numerical Example

Given:

- B = 2000
- w = 4 bytes
- Find address of A[3]

$$= 2000 + (3 \times 4)$$

$$= 2000 + 12$$
$$= 2012$$

🔷 2️⃣ Derivation of Index Formula for 2D Array in C

Problem: Derive address of A[2][3] in row-major and column-major form in a matrix of A[3][4] of floats having base address as 1000.

Let us derive **step-by-step** the address of **A[2][3]** in both **row-major** and **column-major** order.
Given:
int A[3][4];
Where:

- Number of rows (m) = 3
- Number of columns (n) = 4
- Base address (B) = 1000
- Size of each element (w) = 4 bytes
- Lower bound = 0 (as in C)

We want to find the address of:

$$A[2][3]$$

🔷 1️⃣ Row-Major Order (Used in C)

(Row-major is used in C)

✅ Step 1: Understand Storage Order

Row 0 → A[0][0], A[0][1], A[0][2], A[0][3]
Row 1 → A[1][0], A[1][1], A[1][2], A[1][3]
Row 2 → A[2][0], A[2][1], A[2][2], A[2][3]
Entire rows are stored first.

✅ Step 2: Count Elements Before Row 2

Each row has 4 elements.
Before row 2, there are 2 full rows:

$$2 \times 4 = 8 \text{ elements}$$

✅ Step 3: Count Elements Before Column 3 in Row 2

In row 2:
A[2][0], A[2][1], A[2][2]
So 3 elements before A[2][3].

✅ Step 4: Total Elements Before A[2][3]

$$8 + 3 = 11 \text{ elements}$$

✅ Step 5: Convert Elements to Bytes

$$11 \times 4 = 44 \text{ bytes}$$

✅ **Step 6: Add to Base Address**

$$1000 + 44 = 1044$$

✅ **Final Answer (Row-Major)**

$$\boxed{Address = 1044}$$

🔷 2️⃣ **Column-Major Order**

(Column-major is used in Fortran and MATLAB)

✅ **Step 1: Understand Storage Order**

Column 0 → A[0][0], A[1][0], A[2][0]
Column 1 → A[0][1], A[1][1], A[2][1]
Column 2 → A[0][2], A[1][2], A[2][2]
Column 3 → A[0][3], A[1][3], A[2][3]
Entire columns are stored first.

✅ **Step 2: Count Elements Before Column 3**

Each column has 3 elements.
Before column 3, there are 3 full columns:

$$3 \times 3 = 9 \text{ elements}$$

✅ **Step 3: Count Elements Before Row 2 in Column 3**

In column 3:
A[0][3], A[1][3]
So 2 elements before A[2][3].

✅ **Step 4: Total Elements Before A[2][3]**

$$9 + 2 = 11 \text{ elements}$$

✅ **Step 5: Convert to Bytes**

$$11 \times 4 = 44 \text{ bytes}$$

✅ **Step 6: Add to Base Address**

$$1000 + 44 = 1044$$

✅ **Final Answer (Column-Major)**

$$\boxed{Address = 1044}$$

🔷 **Important Observation**

For this particular example:

- Address is SAME in both cases
- But internal counting logic is DIFFERENT

Why same?

Because:

$$(i \times n + j) = (j \times m + i)$$
$$(2 \times 4 + 3) = (3 \times 3 + 2)$$
$$11 = 11$$

🔷 **Final Comparison Table**

Storage	Elements Skipped Logic
Row Major	Skip rows first, then columns
Column Major	Skip columns first, then rows

Generalised Derivation of Index Formula for 2D array

Assume: int A[m][n];
C stores 2D arrays in **Row-Major Order**.

🔷 **What is Row-Major Order?**

In C:

- Entire first row is stored
- Then second row
- Then third row

Example:
int A[3][4];
Memory layout:
Row 0 → A[0][0], A[0][1], A[0][2], A[0][3]
Row 1 → A[1][0], A[1][1], A[1][2], A[1][3]
Row 2 → A[2][0], A[2][1], A[2][2], A[2][3]

🔷 **Derivation of 2D Index Formula (Row-Major)**

Let:

- Base address = B
- Element size = w
- Number of columns = n

- Target element = A[i][j]
- Lower bounds = LBR (row), LBC (column)

🔷 **Step-by-Step Logic**

To reach A[i][j]:

Step 1: Count elements before ith row

Each row has n elements.

Elements before row i:

$$(i - LBR) \times n$$

Step 2: Add elements before column j in same row

$$(j - LBC)$$

Step 3: Total elements before A[i][j]

$$(i - LBR) \times n + (j - LBC)$$

Step 4: Multiply by element size

$$[(i - LBR) \times n + (j - LBC)] \times w$$

✅ **Final Formula (General)**

$$\textbf{Address}(A[i][j]) = B + [(i - LBR) \times n + (j - LBC)] \times w$$

🔷 **In C (Lower Bound = 0)**

Since in C:

- LBR = 0
- LBC = 0

Formula becomes:

$$\textbf{Address}(A[i][j]) = B + (i \times n + j) \times w$$

🔷 **Numerical Example**

Given:

int A[3][4];

- B = 1000
- w = 4 bytes
- n = 4 columns
- Find address of A[2][3]

Step 1: Apply formula

$$= 1000 + (2 \times 4 + 3) \times 4$$
$$= 1000 + (8 + 3) \times 4$$

$$= 1000 + 11 \times 4$$
$$= 1000 + 44$$
$$= 1044$$

🔷 **Why n (columns) is Used?**

Because in **row-major order**, rows are stored completely.
So to reach row i:

- You must skip i × n elements.

🔷 **Derivation of Index Formula for 2D Array in Column-Major Order**

⚠️ Important: C uses **row-major order**.
Column-major order is used in languages like **Fortran** and tools like **MATLAB**.

🔷 **What is Column-Major Order?**

In column-major storage:

- Entire **first column** is stored first
- Then second column
- Then third column

🔷 **Example**

Suppose:
int A[3][4];
Matrix form:

$$\begin{bmatrix} A[0][0] & A[0][1] & A[0][2] & A[0][3] \\ A[1][0] & A[1][1] & A[1][2] & A[1][3] \\ A[2][0] & A[2][1] & A[2][2] & A[2][3] \end{bmatrix}$$

📌 **Memory Layout in Column-Major**

Stored as:
Column 0 → A[0][0], A[1][0], A[2][0]
Column 1 → A[0][1], A[1][1], A[2][1]
Column 2 → A[0][2], A[1][2], A[2][2]
Column 3 → A[0][3], A[1][3], A[2][3]

🔷 **Derivation of Column-Major Formula**

Let:

- Base address = **B**
- Element size = **w**
- Number of rows = **m**
- Target element = **A[i][j]**
- Lower bounds = LBR (row), LBC (column)

✅ **Step 1: Count Elements Before Column j**

Each column has **m elements**.
Number of full columns before column j:

$$(j-LBC)$$

Total elements in those columns:

$$(j - LBC) \times m$$

✅ **Step 2: Add Elements Before Row i in Current Column**

$$(i-LBR)$$

✅ **Step 3: Total Elements Before A[i][j]**

$$(j - LBC) \times m + (i - LBR)$$

✅ **Step 4: Multiply by Element Size**

$$[(j - LBC) \times m + (i - LBR)] \times w$$

✅ **Final General Formula (Column-Major)**

$$\textbf{Address}(\boldsymbol{A[i][j]}) = \boldsymbol{B} + [(\boldsymbol{j} - \boldsymbol{LBC}) \times \boldsymbol{m} + (\boldsymbol{i} - \boldsymbol{LBR})] \times \boldsymbol{w}$$

🔷 **In Case of Lower Bound = 0**

In most languages:

- LBR = 0
- LBC = 0

Formula becomes:

$$\textbf{Address}(\boldsymbol{A[i][j]}) = \boldsymbol{B} + (\boldsymbol{j} \times \boldsymbol{m} + \boldsymbol{i}) \times \boldsymbol{w}$$

🔷 **Numerical Example**

Given:

- A[3][4]
- Base address (B) = 1000
- Element size (w) = 4 bytes
- Find address of A[2][3]

Step 1: Apply Formula

$$\begin{aligned} &= 1000 + (3 \times 3 + 2) \times 4 \\ &= 1000 + (9 + 2) \times 4 \\ &= 1000 + 11 \times 4 \\ &= 1000 + 44 \\ &= 1044 \end{aligned}$$

🔷 Why m (Number of Rows) is Used?

Because in column-major order:

- Columns are stored completely.
- To skip one column, you skip **m elements**.

So:

- Row-major → multiply by **number of columns (n)**
- Column-major → multiply by **number of rows (m)**

🔷 Quick Comparison

Storage Type	**Address Formula**
Row Major	$B + (i \times n + j) \times w$
Column Major	$B + (j \times m + i) \times w$
